Reviewer Pack — Gauss-Kuzmin Deviation Bounds Truth-Table Lempel-Ziv Complex…

Entry b81e6538851e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 09:02:30 UTC

Gauss-Kuzmin Deviation Bounds Truth-Table Lempel-Ziv Complexity

Entry ID: `b81e6538851e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 09:02:30 UTC

1. Conjecture

Field A (mathematical branch): Diophantine approximation / Gauss-Kuzmin distributional theory of continued fractions (the log_2(4/3) frequency of partial-quotient 1 and Khinchin-Levy concentration), classically applied to metric number theory and almost never to descriptive complexity **Field B** (complexity object): Time-bounded Kolmogorov complexity K^poly of Boolean truth tables — the meta-complexity object underlying MCSP, accessed via the polynomial-time-computable LZ77 phrase-count proxy LZ(T) which sandwiches K^poly up to logarithmic factors

Statement:

> For a Boolean function f:{0,1}^n -> {0,1} let T_f in {0,1}^{2^n} be its truth table, r_f = sum_{i<2^n} T_f[i] * 2^{-i-1} in [0,1], and r_f = [0; a_1,...,a_K] its finite continued fraction expansion. Define freq_1(f) = (1/min(K,4n)) * #{i <= min(K,4n) : a_i = 1}, the Gauss-Kuzmin deviation GKD(f) = |freq_1(f) - log_2(4/3)|, and LZ(f) = LZ77 phrase count of T_f. Conjecture: for every n >= 6 and every f, GKD(f) * log_2(LZ(f) + 2) <= 4 * sqrt(n), so any single (f,n) with product exceeding 4*sqrt(n) refutes it.

Rationale (proposer's reasoning):

> Gauss-Kuzmin governs partial-quotient statistics of almost every irrational, but a truth table is a specific dyadic rational T_f / 2^{2^n} whose CF expansion is a number-theoretic fingerprint of its descriptive structure. Highly compressible T_f (small LZ ~ K^poly) tend to have arithmetic structure that biases partial-quotient frequencies away from log_2(4/3), while incompressible T_f obey Khinchin-Levy concentration with GKD ~ O(1/sqrt(K)); the conjectured product bound asserts a uniform sqrt(n) trade-off bridging these regimes. The statement is a UNIVERSAL inequality on a product of two computable invariants — neither a largeness nor a usefulness separator — so it sidesteps the natural-proofs barrier that killed the Mahler-measure attempt in this approach.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1384421249ccf520`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 450 instances (n in {6,8,10}; 30 seeds x 5 families), compute GKD(f)*log2(LZ(f)+2) and threshold T(n)=4*sqrt(n). SUPPORTED iff overall pass rate >=95%, every family >=80% pass, and no instance exceeds 1.2*T(n). FALSIFIED iff any single instance has product > 1.2*T(n).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Gauss-Kuzmin continued fraction Kolmogorov complexity truth table - Lempel-Ziv complexity continued fraction partial quotients Boolean - Khinchin Levy LZ77 meta-complexity MCSP truth table`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
from math import log2, sqrt

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_truth_table(n):
        return [random.randint(0, 1) for _ in range(2**n)]

    def truth_table_to_r_f(truth_table):
        r_f = Fraction(0)
        n = len(truth_table)
        for i in range(n):
            if truth_table[i]:
                r_f += Fraction(1, 2**(i+1))
        return r_f

    def continued_fraction_expansion(r_f, K_trunc):
        a = []
        while r_f != 0 and len(a) < K_trunc:
            a.append(int(1 / r_f))
            r_f = Fraction(1 / r_f - int(1 / r_f))
```

```
    return a

def freq_1(f, K_trunc):
    n = len(f)
    count = sum(1 for i in range(min(K_trunc, 4*n)) if f[i] == 1)
    return count / min(K_trunc, 4*n)

def LZ77_phrase_count(truth_table):
    n = len(truth_table)
    phrase_count = 0
    i = 0
    while i < n:
        j = i + 1
        k = i
        while j < n and truth_table[j] == truth_table[k]:
            j += 1
        phrase_count += 1
        if j - i > 1:
            phrase_count += LZ77_phrase_count(truth_table[i+1:j])
        i = j
    return phrase_count

def generate_random_k_DNF(n, k):
    variables = list(range(n))
    terms = []
    for _ in range(k):
        term = random.sample(variables, random.randint(1, n))
        terms.append(term)
    return terms

def evaluate_k_DNF(truth_table, k_DNF):
    n = len(truth_table)
    m = len(k_DNF)
    result = [0] * n
    for i in range(n):
        for j in range(m):
            term = k_DNF[j]
            if all(truth_table[i + var] == 1 for var in term):
                result[i] = 1
                break
    return result

def generate_dictator(n, variable):
    return [int(i // (2**(n-variable-1)) % 2) for i in range(2**n)]

def generate_parity(n):
    return [sum(i // (2**j) % 2 for j in range(n)) % 2 for i in range(2**n)]

def generate_majority_threshold(n, threshold):
    return [1 if sum(i // (2**j) % 2 for j in range(n)) >= threshold else 0 for i in range(2**

n_values = [6, 8, 10]
instances_tested = 0
total_product = 0
counterexample = ""
freq_1_list = []
```

```
LZ77_list = []

for n in n_values:
    K_trunc = 4 * n

    # Uniform random T_f
    truth_table = generate_truth_table(n)
    r_f = truth_table_to_r_f(truth_table)
    a = continued_fraction_expansion(r_f, K_trunc)
    freq_1_val = freq_1(a, K_trunc)
    LZ77_val = LZ77_phrase_count(truth_table)
    product = freq_1_val * log2(LZ77_val + 2)
    instances_tested += 1
    total_product += product
    if product > 4 * sqrt(n):
        counterexample += "Uniform random T_f, n={n}, product={product}\n"

    # Random k-DNFs of sizes {1, n, n^2}
    for k in [1, n, n**2]:
        truth_table = evaluate_k_DNF(generate_truth_table(n), generate_random_k_DNF(n, k))
        r_f = truth_table_to_r_f(truth_table)
        a = continued_fraction_expansion(r_f, K_trunc)
        freq_1_val = freq_1(a, K_trunc)
        LZ77_val = LZ77_phrase_count(truth_table)
        product = freq_1_val * log2(LZ77_val + 2)
        instances_tested += 1
        total_product += product
        if product > 4 * sqrt(n):
            counterexample += f"Random k-DNF of size {k}, n={n}, product={product}\n"

    # Dictator
    truth_table = generate_dictator(n, random.randint(0, n-1))
    r_f = truth_table_to_r_f(truth_table)
    a = continued_fraction_expansion(r_f, K_trunc)
    freq_1_val = freq_1(a, K_trunc)
    LZ77_val = LZ77_phrase_count(truth_table)
    product = freq_1_val * log2(LZ77_val + 2)
    instances_tested += 1
    total_product += product
    if product > 4 * sqrt(n):
        counterexample += f"Dictator, n={n}, product={product}\n"

    # Parity
    truth_table = generate_parity(n)
    r_f = truth_table_to_r_f(truth_table)
    a = continued_fraction_expansion(r_f, K_trunc)
    freq_1_val = freq_1(a, K_trunc)
    LZ77_val = LZ77_phrase_count(truth_table)
    product = freq_1_val * log2(LZ77_val + 2)
    instances_tested += 1
    total_product += product
    if product > 4 * sqrt(n):
        counterexample += f"Parity, n={n}, product={product}\n"

    # Majority/Threshold
    threshold = random.randint(1, n//2)
```

```
        truth_table = generate_majority_threshold(n, threshold)
        r_f = truth_table_to_r_f(truth_table)
        a = continued_fraction_expansion(r_f, K_trunc)
        freq_1_val = freq_1(a, K_trunc)
        LZ77_val = LZ77_phrase_count(truth_table)
        product = freq_1_val * log2(LZ77_val + 2)
        instances_tested += 1
        total_product += product
        if product > 4 * sqrt(n):
            counterexample += f"Majority/Threshold, n={n}, threshold={threshold}, product={product

    mean_product = total_product / instances_tested
    support_fraction = (instances_tested - len(counterexample.split('\n'))) / instances_tested

    return {
        "metric_name": "GKD(f) * log2(LZ(f) + 2)",
        "metric_value": mean_product,
        "instances_tested": instances_tested,
        "conjecture_holds": support_fraction >= 0.8 and len(counterexample.split('\n')) == 0,
        "counterexample": counterexample.strip()
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_product = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if not r["counterexample"]) / len(results)

    if all(not r["counterexample"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_product} std=0.0 support_fraction={support_fraction}'
    elif any(r["metric_value"] > 1.2 * 4 * sqrt(n) for n, r in zip([6, 8, 10], results)):
        first_failing_seed = seeds[next(i for i, r in enumerate(results) if r["metric_value"] > 1.
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0501520.py", line 177, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0501520.py", line 112, in run_trial
```

```
    truth_table = evaluate_k_DNF(generate_truth_table(n), generate_random_k_DNF(n, k))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0501520.py", line 74, in evaluate_k_DN
    if all(truth_table[i + var] == 1 for var in term):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0501520.py", line 74, in <genexpr>
    if all(truth_table[i + var] == 1 for var in term):
           ~~~~~~~~~~~^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an IndexError in evaluate_k_DNF before producing any RESULT line, so neither the support nor falsification criteria could be evaluated. | next: Fix the indexing bug in evaluate_k_DNF (likely the i + var truth-table lookup) and rerun the 450-instance pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	168329
2	propose	claude_max	opus	0	0	398503
3	propose	claude_max	opus	0	0	268606
4	preregistration	claude_max	opus	0	0	7863
5	novelty	claude_max	opus	0	0	3226
6	novelty	claude_max	opus	0	0	7039
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19606
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21418
9	judge	claude_max	opus	0	0	4538

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 899127 ms total latency. Provider mix: {'claude_max': 7, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/b81e6538851e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b81e6538851e.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b81e6538851e.tar.gz` (if generated)